

面向对象程序设计

第02章 数据类型和引用

赵毅力 ylzhao@swfu.edu.cn

大数据与智能工程学院

西南林业大学

Java变量的分类

按声明的位置划分

局部变量：方法或语句块内定义的变量

成员变量：方法外或类的内部定义的变量

按所属的数据类型划分

基本类型变量

引用类型变量

Java语言的基本数据类型

数据类型	含义	类别
byte	有符号8位整数	值类型
short	有符号16位整数	值类型
int	有符号32位整数	值类型
long	有符号64位整数	值类型
float	32位浮点数	值类型
double	64位浮点数	值类型
char	字符类型	值类型
boolean	布尔类型	值类型

整数类型

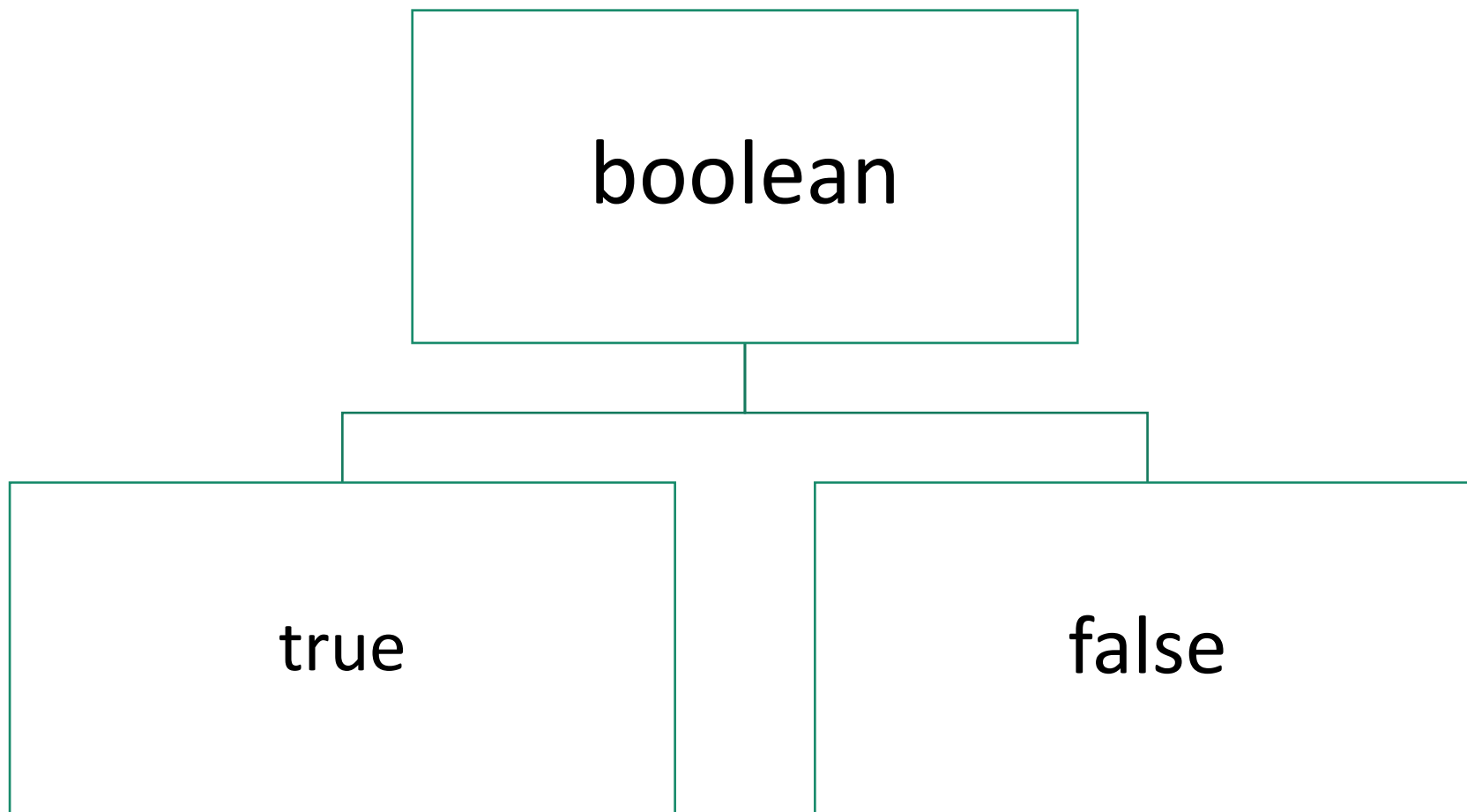
小数类型

备注：Java语言没有无符号类型。

Java引用数据类型

数据类型	名称	类别
数组	[]	引用类型
类	class	引用类型
枚举	enum	引用类型
接口	interface	引用类型
字符串	String	引用类型

布尔类型



字符类型 - char

- 字符常量：用**单引号**括起来的单个字符。
 - 例如：'c'，'E'或者'中'。
- Java 字符采用**UTF-16**编码的Unicode字符，每个字符使用两个字节表示。
 - 转义字符'\'（和C语言类似）
 - '\n'：换行
 - '\"'：表示单个引号'
 - '\\': 表示单个反斜杠\

运算符

- 算术运算符: +, -, *, /, %, ++, --
- 关系运算符: >, <, >=, <=, ==, !=
- 逻辑运算符: &&, ||, !
- 赋值运算符: =, +=, -=, *=, /=
- 字符串连接运算符: +
 - String key = "key1";
 - String value = "value1";
 - String prop = key + value;

条件分支语句 (1)

```
if (布尔表达式) {  
    .....  
}
```

```
if (布尔表达式) {  
    .....  
} else {  
    .....  
}
```

条件分支语句 (2)

```
if (布尔表达式1) {  
    .....  
} else if (布尔表达式2) {  
    .....  
}
```

```
if (布尔表达式1) {  
    .....  
} else if (布尔表达式2) {  
    .....  
} else {  
    .....  
}
```

例子

- 计算某一年是否是闰年
 - 要么能够被4整除，但不能被100整除，例如2008;
 - 要么能够被400整除，例如2000。
- 使用变量year表示某一个年份

条件分支语句：switch-case

```
switch (整型表达式) {  
    case 整型常量1:  
        ...  
    case 整型常量2:  
        ...  
    ...  
    default:  
        ...  
}
```

一个小例子

```
int index = 3;  
switch (index) {  
    case 1: System.out.println("1");  
    case 2: System.out.println("2");  
    case 3: System.out.println("3");  
    case 4: System.out.println("4");  
    default: System.out.println("5");  
}
```

输出什么？

循环控制

for循环

while循环

do-while循环

for 循环

```
for (表达式1; 表达式2; 表达式3) {  
    .....  
}
```

例子

- $1 + 3 + 5 + 7 + \dots + 99$

while 循环 / do-while 循环

```
while (布尔表达式) {  
    ...  
}
```

- 只要布尔表达式为true，循环中的程序代码就会一直重复执行。

```
do {  
    ...  
} while (布尔表达式)
```

- 首先执行循环中的语句，然后判断布尔表达式的值。
- 如果布尔表达式为真，就继续重复执行循环中的语句；否则退出循环。

break 语句

- break 语句用于在循环中强行跳出循环语句。

```
int flag = 4;  
for (int i = 1; i <= 10; i++) {  
    if (i == flag) {  
        break;  
    }  
  
    System.out.println("i = " + i);  
}
```

输出什么？

```
i = 1  
i = 2  
i = 3
```

continue 语句

- continue 语句用于在循环体中提前终止当前循环过程，跳过 continue 语句后面所有的语句，直接开始下一次循环。

```
int flag = 4;  
for (int i = 1; i <= 5; i++) {  
    if (i == flag) {  
        continue;  
    }  
  
    System.out.println("i = " + i);  
}
```

输出什么？

i	=	1
i	=	2
i	=	3
i	=	5

数组概述

数组是引用类型

数组创建后大小不可以改变

数组可以是一维或多维的。

数值数组元素的默认值设置为**0**，而引用元素的默认值设置为 **null**。

数组的索引从**0**开始：具有 n 个元素的数组的索引是从 **0** 到 **$n-1$** 。

一维数组

声明一个整型数组变量：

- `int[] nums; // nums的值是多少?`

创建数组

- `nums = new int[7];`

为数组的每一个元素进行赋值：

- `nums[0] = 0;`
- `nums[1] = nums[0];`

创建数组并且初始化

- `int[] nums = new int[] {2, 4, 6};`
- `int[] nums = {2, 4, 6, 8, 10};`

如何知道数组的大小

- Java语言中的一维数组对象都有一个成员：`length`
- `数组名.length`就表示该数组对象的元素个数。

例子

- 斐波那契数列

$$F(n) = \begin{cases} 1, n = 1 \\ 1, n = 2 \\ F(n - 1) + F(n - 2), n > 2 \end{cases}$$

使用增强的for循环对数组元素进行迭代

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };  
for (int i : numbers) {  
    System.out.print(i);  
}  
// Output: 4 5 6 1 2 3 -2 -1 0
```

二维数组

创建一个整型二维数组变量：

- `int[][] nums2D = new int[2][3];`
- `int[][] nums2D = {{1, 2, 3}, {4, 5, 6}};`

二维数组对应于线性代数中的矩阵

- 用二维数组实现矩阵转置

数组和集合

Java语言中的数组也是对象，定义后不能改变其大小。

Java的标准库中包含了很多复杂的数据结构，如动态数组和哈希表等。

如果需要快速、有序地对元素进行排序时，数组是不错的选择。

数组能够让你使用位置索引来快速、随机地访问其中的元素。

如果需要对容器进行动态改变，应该使用集合类型。

项目实例1

- 根据三角形的三条边 e_1 、 e_2 、 e_3 ，计算对应的三角形面积。
- 已知三角形的面积计算公式为：

$$area = \sqrt{s(s - e_1)(s - e_2)(s - e_3)}$$

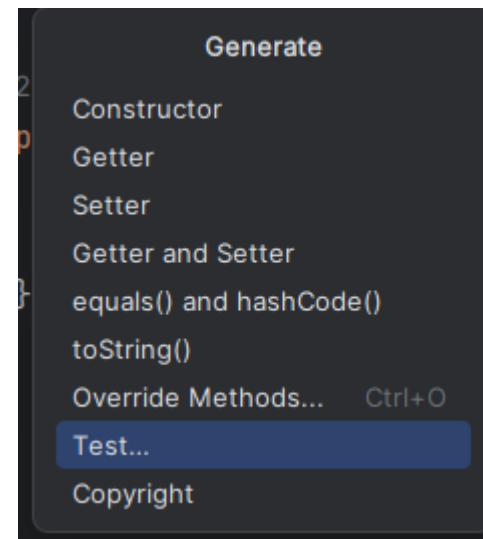
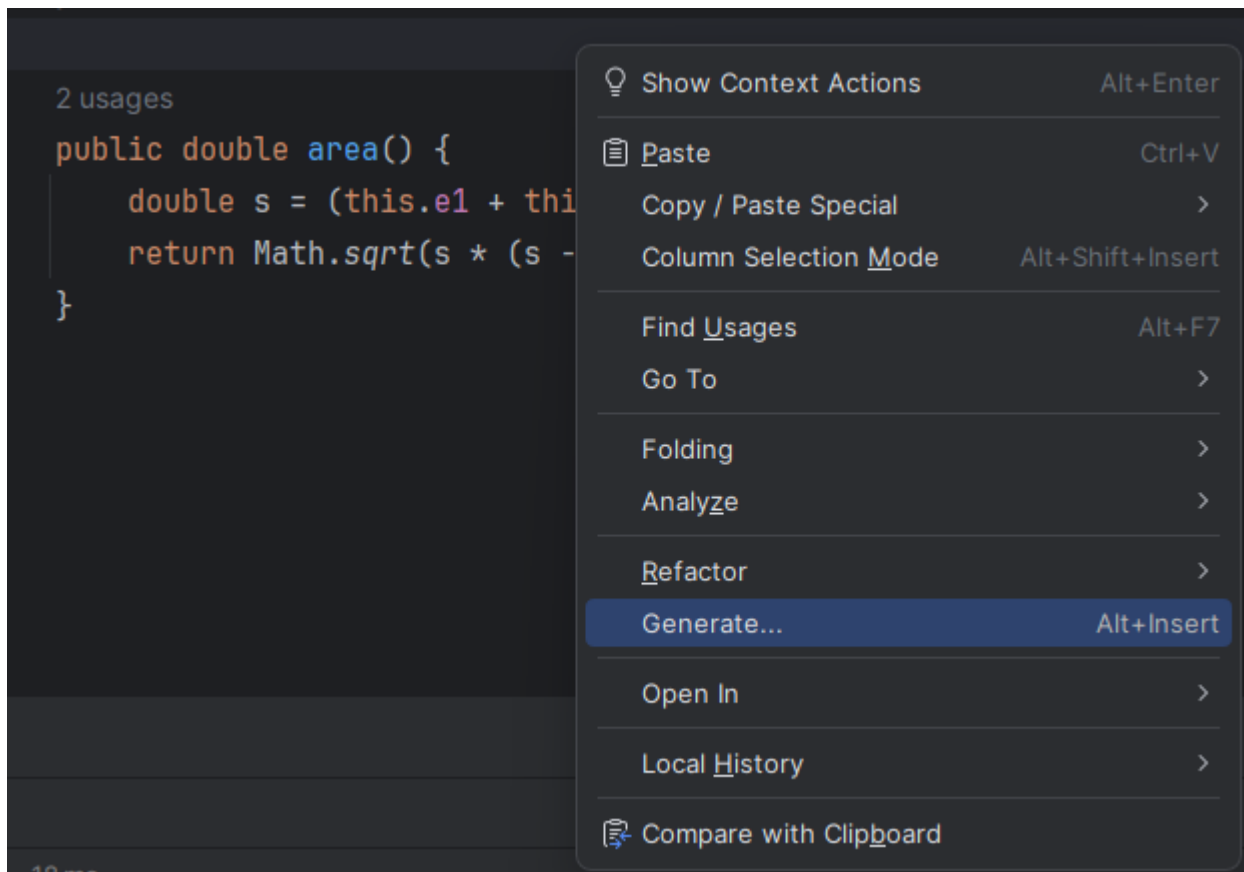
其中 $s = (e_1 + e_2 + e_3)/2$ 。

项目实例1-单元测试

- 如何判断计算三角形面积的实现是正确的？
- Java项目中常用的一种方法是：**单元测试**
- 可以通过开源的框架Junit完成单元测试。

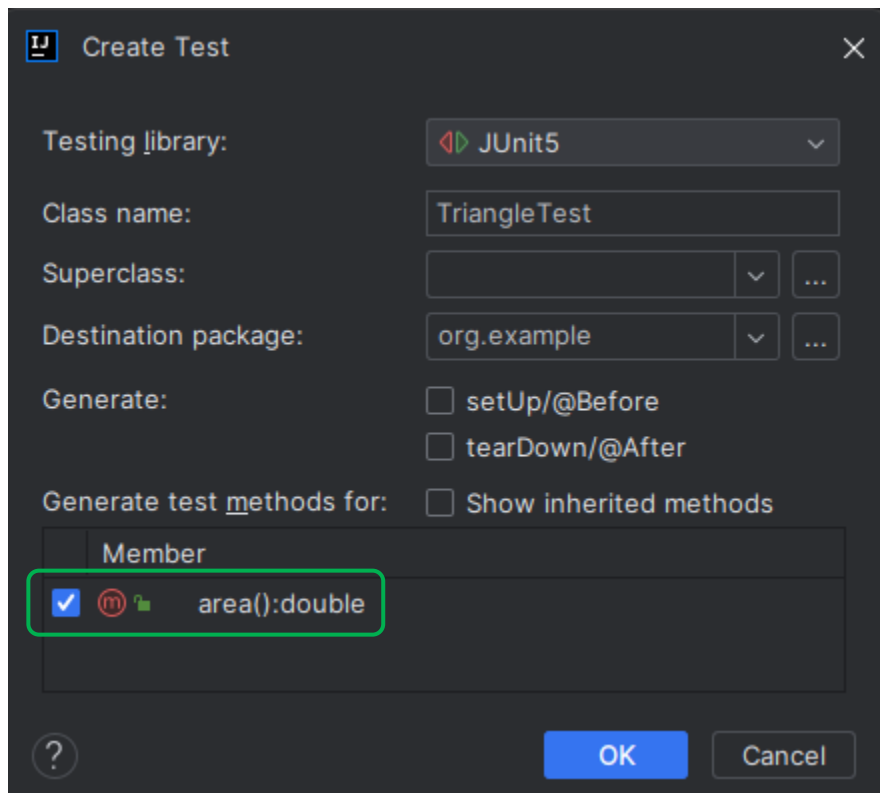
对需要测试的方法添加单元测试

- 在需要添加单元测试的类代码空白处单击鼠标右键，选择“Generate...”菜单选项，然后选择“Test...”选项。



对需要测试的方法添加单元测试

- 在“Create Test”对话框中选择需要进行单元测试的方法。



使用JUnit完成单元测试

- 在项目的pom.xml中添加JUnit项目的依赖

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

使用JUnit完成单元测试

- 通过@Test注解实现单元测试

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TriangleTest {
    @Test
    void area() {
        double[] es = { 2.7, 3.2, 1.5 };
        double[] rs = { 2.0174241001832014 };
        Triangle triangle = new Triangle(es[0], es[1], es[2]);
        double s = triangle.area();
        assertEquals(rs[0], s, 1e-5);
    }
}
```

使用JUnit完成单元测试

- 为前面的斐波那契数列项目添加单元测试。
- 需要考虑如何为数组类型添加单元测试。